

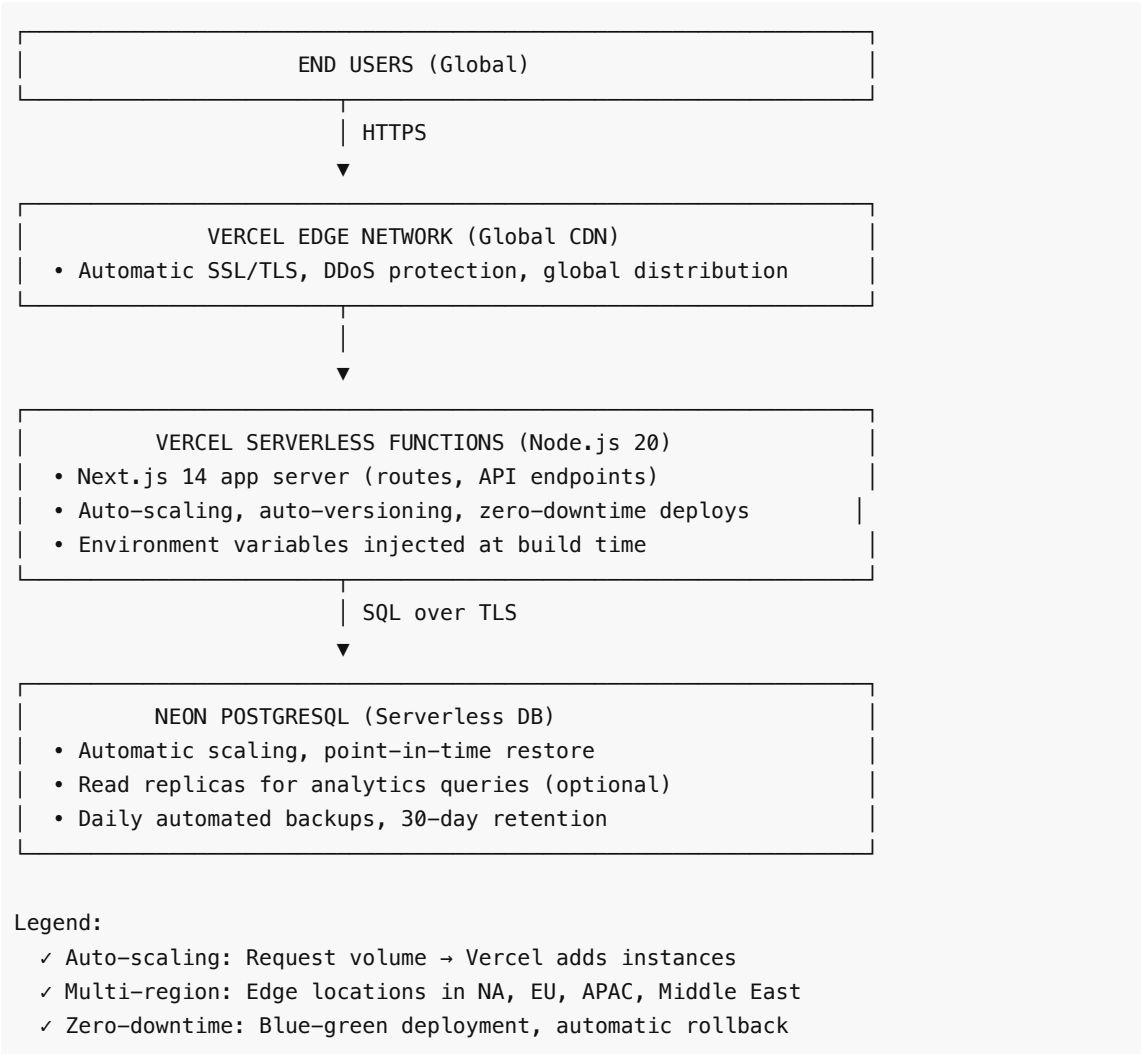
IPOReady Production Deployment Guide

Complete Step-by-Step Deployment Reference

For: Development team, DevOps, deployment verification

Date: June 1, 2026 Version: 1.0 (Vercel + Neon PostgreSQL Stack)

Deployment Architecture Overview



Pre-Deployment Checklist

Before deploying to production, verify:

Code Quality

- ☐ All tests passing: `npm run test`
- ☐ No TypeScript errors: `npm run build`
- ☐ No console errors in development: `npm run dev`
- ☐ Linting passes: `npm run lint`
- ☐ Security audit clean: `npm audit`

Database

- ☐ All migrations run on staging: `npm run migrate:staging`
- ☐ Database schema matches code expectations
- ☐ No breaking changes to existing tables
- ☐ Rollback plan documented (see section "Rollback Procedures")

Secrets & Configuration

- ☐ All environment variables set in Vercel project settings
- ☐ Database connection string verified in staging
- ☐ API keys rotated (if deploying new integrations)
- ☐ CORS configuration correct for production domain
- ☐ JWT secret updated (if applicable)

Third-Party Integrations

- ☐ Stripe API keys correct for production
- ☐ Email service (SendGrid/Mailgun) configured for prod domain
- ☐ Slack webhook URLs updated (if using notifications)
- ☐ Google OAuth credentials pointing to production domain
- ☐ Any webhooks point to production domain

Performance Baselines

- ☐ Homepage loads in < 2 seconds (Lighthouse test)
- ☐ API endpoints respond in < 500ms (p95 latency)
- ☐ Database queries use indexes (no full table scans on large queries)
- ☐ No N+1 query problems in critical paths

Monitoring & Alerts

- ☐ Error tracking set up (e.g., Sentry)
 - ☐ Log aggregation configured (e.g., Vercel logs)
 - ☐ Health check endpoint verified: `curl https://ipoready.com/api/health`
 - ☐ Uptime monitoring enabled (e.g., Datadog, PagerDuty)
 - ☐ On-call rotation established
-

Step 1: Deploy via Vercel Dashboard (Recommended)

Via Git Push to Main Branch

The easiest way—automatic deployment on every commit to main:

1. Ensure your code is committed and pushed to GitHub:

```
git add .  
git commit -m "Feature: [description of change]"  
git push origin main
```

2. Vercel automatically detects the push and starts deployment:

- Vercel rebuilds the Next.js app
- Runs your build command: `npm run build`
- Deploys to edge network
- Old version remains live until new version passes health checks
- **Zero downtime** (blue-green deployment)

3. Monitor deployment progress:

- Visit Vercel dashboard: <https://vercel.com/ipoready>
- Click the latest deployment
- Watch the build log for errors
- Deployment completes in 2-3 minutes

4. Verify deployment succeeded:

- Check deployment status badge shows  "Ready"
 - Visit <https://ipoready.com> (production domain)
 - Confirm you see the latest UI changes
 - Hit health check endpoint: <https://ipoready.com/api/health>
-

Step 2: Manual Deployment via CLI (Advanced)

When you need to deploy outside the Git workflow or redeploy an older commit:

Prerequisites

```
# Install Vercel CLI (one-time)
npm install -g vercel

# Login to Vercel account
vercel login

# Follow the browser prompt to authenticate
```

Deploy Current Branch

```
# Deploy the current branch to staging
vercel --prod

# The CLI will:
# 1. Build the Next.js app locally
# 2. Upload compiled code to Vercel
# 3. Deploy to production
# 4. Print deployment URL and status
```

Deploy Specific Commit

```
# Deploy the code from an older commit
git checkout abc1234 # Your target commit hash
vercel --prod       # Deploy that commit
git checkout main   # Return to main branch
```

Examples

```
# Deploy after emergency hotfix
git commit -m "Fix: health check bug"
vercel --prod
# Takes ~2-3 minutes; prints: Deployed to https://ipoready.com

# Rollback to previous version
git log --oneline | head -5 # Find the previous good commit
git checkout 7c9f3a2        # The previous commit
vercel --prod
git checkout main
```

Step 3: Database Migration (Pre-Deployment)

Run any schema changes BEFORE deploying the app code.

Check for Pending Migrations

```
# List migrations that haven't run yet
npm run migrate:status

# Output example:
# Pending migrations:
#   - 010_complete_cap_table_schema.sql
#   - 011_add_audit_logging.sql
```

Run Migrations on Production

```
# Set database URL to production
export DATABASE_URL="postgresql://user:pass@host/dbname"

# Run all pending migrations
npm run migrate:prod

# Output example:
# Running migration: 010_complete_cap_table_schema.sql
# Creating table: share_classes_v2 ✓
# Creating table: shareholders ✓
# Creating indexes ✓
# Completed in 4.2 seconds
```

Verify Migration Success

```
# Connect to production database
psql $DATABASE_URL

# List tables (verify new tables exist)
\dt

# Check table structure
\d share_classes_v2

# Verify row counts
SELECT table_name, row_count FROM (
  SELECT table_name, count(*) as row_count FROM information_schema.tables
  WHERE table_schema = 'public'
) GROUP BY table_name;

# Exit psql
\q
```

Important: Migrations Are Irreversible

- **Rollback is NOT automatic.** If a migration fails, you must:
 1. Fix the SQL in the migration file
 2. Increment the version number (010 → 011)
 3. Re-run `npm run migrate:prod`
 - **Do NOT modify a migration after it's run.** Create a new migration instead.
-

Step 4: Environment Variables

Vercel automatically injects these at build and runtime.

Core Environment Variables

Set these in Vercel project settings (Settings > Environment Variables):

```
# Database (Neon PostgreSQL)
DATABASE_URL="postgresql://user:password@host/database?schema=public"

# NextAuth (authentication)
NEXTAUTH_SECRET="your-secret-key-here" # Generate: openssl rand -base64 32
NEXTAUTH_URL="https://ipoready.com"

# Stripe (payments)
STRIPE_PUBLIC_KEY="pk_live..."
STRIPE_SECRET_KEY="sk_live..."
STRIPE_WEBHOOK_SECRET="whsec..."

# Email Service (SendGrid/Mailgun)
SENDGRID_API_KEY="SG.xxx..."
# OR
MAILGUN_API_KEY="key-xxx..."
MAILGUN_DOMAIN="mail.ipoready.com"

# Google OAuth
GOOGLE_CLIENT_ID="...apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="..."

# Slack Integration
SLACK_BOT_TOKEN="xoxb-..."
SLACK_SIGNING_SECRET="..."

# Logging & Monitoring
SENTRY_DSN="https://xxx@xxx.ingest.sentry.io/123"
LOG_LEVEL="info" # or "debug" for troubleshooting
```

Environment-Specific Variables

Set these differently for staging vs. production:

Variable	Staging	Production
APP_ENV	staging	production
NEXTAUTH_URL	https://staging.ipoready.com	https://ipoready.com
LOG_LEVEL	debug	info
DATABASE_URL	Staging DB	Production DB

How to Set Variables in Vercel

1. Go to Vercel dashboard → Project → Settings
 2. Click "Environment Variables"
 3. For each variable:
 - Name: `DATABASE_URL`
 - Value: `postgresql://...`
 - Select environments: Production, Preview, Development (or your preference)
 - Click "Add"
 4. Click "Save"
 5. Redeploy for variables to take effect: `vercel --prod`
-

Step 5: Health Check & Verification

After deployment, verify the app is alive and responding correctly.

Quick Health Check (30 seconds)

```
# Hit the health check endpoint
curl https://ipoready.com/api/health

# Expected response (HTTP 200):
{
  "status": "healthy",
  "timestamp": "2026-06-01T12:00:00Z",
  "database": "connected",
  "services": {
    "auth": "operational",
    "email": "operational",
    "stripe": "operational"
  }
}
```

Detailed Health Diagnostics

```
# Check all services are operational
curl -s https://ipoready.com/api/health | jq '.services'

# Check database connectivity
curl -s https://ipoready.com/api/health | jq '.database'

# Check error rates (last hour)
curl -s https://ipoready.com/api/metrics/errors

# Expected: { "errorCount": 0, "period": "1h" }
```

Manual User Flow Tests (2-3 minutes)

Verify critical user journeys work:

1. Sign In Flow

- Go to <https://ipoready.com>
- Click "Sign In" → "Google"
- Authorize with test Google account
- Verify you see your company dashboard
-  Check: Dashboard loads, PACE™ score visible

2. Cap Table Functionality

- Go to Cap Table > View Current
- Verify shareholder list loads
- Click a shareholder → verify vesting schedule visible

-  Check: No errors in browser console

3. Document Upload

- Go to Documents > Upload
- Try uploading a PDF (or dummy file)
- Verify upload completes
-  Check: File appears in document list

4. Email Notification

- Invite a team member via email
- Check your inbox (within 2 minutes)
- Click the invitation link
- Verify you can accept it
-  Check: Email arrives and link works

5. Database Query Performance

- Open DevTools (F12) → Network tab
 - Reload the dashboard
 - Click through a few features (Cap Table, Documents, Team)
 - Verify API responses < 500ms
 -  Check: No "waterfall" of slow requests
-

Troubleshooting Deployment Issues

Issue: Build Fails with "TypeScript Errors"

Symptom: Vercel deployment fails with error like "Cannot find module 'X'"

Solution:

```
# Verify locally first
npm run build

# Fix TypeScript errors shown in output
# Common issues:
#   - Missing type definitions: npm install --save-dev @types/xyz
#   - Import path wrong: Check relative paths, use tsconfig paths
#   - Env variable missing: Add to .env.local

# Commit and push
git add .
git commit -m "Fix: TypeScript build errors"
git push origin main
```

Issue: Deployment Succeeds but App Shows 500 Error

Symptom: <https://ipoready.com> loads but shows "500 Internal Server Error"

Solution:

```
# 1. Check Vercel deployment logs
#   Vercel Dashboard > Deployments > Latest > Logs

# 2. Check error tracking (if Sentry is configured)
#   https://sentry.io/organizations/ipoready/issues/

# 3. Verify environment variables are set
#   Vercel Dashboard > Settings > Environment Variables
#   Confirm DATABASE_URL, STRIPE keys, etc. are present

# 4. Check database connectivity
curl https://ipoready.com/api/health | jq '.database'
#   If "database": "disconnected", DATABASE_URL is wrong

# 5. Rollback to previous deployment (if recent change broke it)
#   Vercel Dashboard > Deployments > Find last good deployment > Promote to
Production
```

Issue: Database Connections Exceed Limit

Symptom: Errors like "too many connections" or "connection timeout"

Solution:

```
# 1. Check Neon database limits
#   Neon Console > Project > Branches > main > Pooling
#   Verify pool size is appropriate

# 2. Verify your connection string uses connection pooling
#   DATABASE_URL should include "pooling=session" or "pooling=transaction"
#   Example: postgresql://user:pass@host/db?pooling=session

# 3. Check for connection leaks in code
#   Search for: database.query() without proper cleanup
#   All queries should use the shared pool, not individual connections

# 4. Increase Neon pool size (if warranted)
#   Neon > Project Settings > Connection Pooling > Increase pool_size
```

Issue: Deployment Takes > 5 Minutes or Times Out

Symptom: Vercel log shows "Build timeout" or > 600 seconds

Solution:

```
# 1. Check build log for slow steps
#   Vercel Dashboard > Latest Deployment > Logs
#   Look for slow npm install, slow test runs, or large asset builds

# 2. Optimize dependencies
#   # Remove unused packages
#   npm ls # Shows dependency tree
#   npm uninstall unused-package
#   # Check for duplicate transitive deps
#   npm dedupe

# 3. Disable slow build processes
#   # Comment out slow tests or linters from package.json "build" script
#   # Move heavy tasks to separate "postbuild" hook if needed

# 4. Increase Vercel build timeout
#   Vercel Dashboard > Settings > Build Timeout
#   Change from 600s to 900s or 1200s
```

Rollback Procedures

If production deployment has critical bugs, roll back to the previous version.

Immediate Rollback (< 2 minutes)

1. Via Vercel Dashboard (Recommended)

- Go to <https://vercel.com/ipoready>
- Find "Deployments" section
- Locate the last **working** deployment (marked  Ready)
- Click "..." (more options) → "Promote to Production"
- Confirm the rollback
- **Done** — production is now running the previous version

2. Via Git & Vercel CLI

```
# Find the commit hash of the last good deploy
git log --oneline | head -5
# Example output:
#   abc1234 Fix: cap table validation
#   def5678 Add PACE scoring (BAD - this one broke things)
#   ghi9012 Refactor database schema

# Checkout the good commit
git checkout abc1234

# Redeploy
vercel --prod

# Return to main
git checkout main
```

After Rollback: Fix & Redeploy

1. **Identify the bug** (in the rolled-back version)
2. **Create a fix commit** on main branch:

```
git checkout main
# Fix the code
git add .
git commit -m "Fix: [describe the bug you fixed]"
git push origin main
```

3. Redeploy when ready:

```
# Vercel auto-deploys on git push
# Or manually: vercel --prod
```

Monitoring Production

Daily Checks (5-10 minutes)

```
# Every morning, verify:

# 1. App is up
curl https://ipoready.com/api/health

# 2. No spike in errors
curl https://ipoready.com/api/metrics/errors?period=24h

# 3. Database is responsive
curl https://ipoready.com/api/metrics/database

# 4. No scheduled maintenance windows (Neon, Stripe, etc.)
# Check Neon: https://status.neon.tech/
# Check Stripe: https://status.stripe.com/
# Check Vercel: https://vercel-status.com/
```

Weekly Report

Every Monday: Generate a production health report:

```
# Get metrics from last 7 days
curl -s https://ipoready.com/api/metrics/weekly-summary | jq '.'

# Expected output:
{
  "period": "7d",
  "uptime_pct": 99.95,
  "error_count": 12,
  "avg_response_time_ms": 245,
  "database_avg_latency_ms": 85,
  "top_errors": [
    { "endpoint": "/api/pace/scores", "count": 8 },
    { "endpoint": "/api/team/invite", "count": 4 }
  ]
}
```

Monthly Capacity Review

First week of each month: Check if you're approaching any limits:

- **Vercel:** Check function invocations, bandwidth, build minutes
 - Vercel Dashboard > Usage
- **Neon:** Check database size, connection count, query performance
 - Neon Console > Project > Branches > main > Usage
- **Stripe:** Check API rate limits, failed payments

- Stripe Dashboard > Logs > API Requests
 - **SendGrid/Mailgun:** Check email delivery rate, bounces
 - Service Dashboard > Reports
-

Security Checklist (Post-Deployment)

After deploying to production, verify these security measures:

- ☐ SSL/TLS enabled (HTTPS only)
 - Verify: `curl -I https://ipoready.com | grep Strict-Transport-Security`
 - Expected: `Strict-Transport-Security: max-age=31536000`
- ☐ Environment secrets NOT in code
 - Run: `git log -p | grep -i "secret\|password\|api.key"`
 - Expected: No matches (secrets only in Vercel)
- ☐ Database encryption in transit
 - DATABASE_URL should include `sslmode=require`
- ☐ CORS configured correctly
 - Verify: Only your domain can access the API
 - Check: `/api/health` is public, `/api/admin/*` is restricted
- ☐ Rate limiting enabled
 - Verify: `/api/auth/login` allows max 10 requests per minute per IP
- ☐ Audit logging active
 - Verify: All data modifications are logged in `audit_log` table
- ☐ Monitoring & alerting set up
 - Alert on: 5xx errors, database connection failures, rate limit spikes

Deployment Checklist Template

Use this before every production deployment:

Pre-Deployment (Day of)

- [] Code reviewed and merged to main
- [] All tests passing locally
- [] Database migrations written (if schema changes)
- [] Environment variables configured in Vercel
- [] Health check endpoint verified in staging

Deployment

- [] Run migrations on production: `npm run migrate:prod`
- [] Push to main or run: `vercel --prod`
- [] Monitor build log in Vercel dashboard
- [] Wait for deployment to mark as  Ready

Post-Deployment (Immediately After)

- [] Run health check: `curl https://ipoready.com/api/health`

- [] Manual user flow test (sign in → cap table → dashboard)
- [] Check error tracking for new errors (Sentry)
- [] Verify database is responding: `curl /api/health | jq '.database'`

Post-Deployment (After 1 Hour)

- [] No spike in error rate
- [] Response times normal (< 500ms p95)
- [] All critical features working for test users
- [] Email notifications being sent correctly

Rollback Plan (If Issues Arise)

- [] Rollback ready: know the last good deployment commit
- [] Rollback executed: vercel promotion to previous deployment
- [] Team notified of rollback reason
- [] Bug fix PR created on main branch

For questions or issues, contact: devops@ipoready.com